RevivalOS - Razorpay AI Revenue Recovery Agent

Complete build plan for Razorpay AI Buildathon 2026, Track 03 - Prepared August 2026

What This Document Is

This is the complete build plan for a Razorpay AI Buildathon 2026 entry in Track 03 - AI Revenue Recovery. It is written so one person can read it end to end and know exactly what we are building, why every piece exists, how the app looks and behaves screen by screen, where the AI agents sit, and how we stand out from every other prompt-and-demo submission. The product has a name: RevivalOS - a Revenue Recovery Control Plane. Everything below is grounded in Razorpay's own documented behaviour and in the official buildathon brief, and it is honest about what is known versus what we must request or simulate.

The one-line thesis

- Given a VERIFIED revenue-risk event, recover the most money with the least customer friction - and prove exactly what happened, why, and when we chose to stop.
- That is narrower, more measurable, and more useful to Razorpay than 'an AI that sends payment reminders.'

The five things a judge should remember

- RevivalOS is a cross-surface control plane - one case model that spans payment failures, abandoned checkouts, failed subscriptions and overdue receivables - not a clone of a single existing agent.
- Deterministic code owns money truth; AI owns bounded reasoning (classify, prioritise, draft, extract, explain). The AI can never move money on its own.
- Every rupee we claim is a VERIFIED capture, reconciled against Razorpay's live state - never inferred from a webhook. Refunds and disputes are subtracted back out.
- Safety is a visible product feature, not hidden plumbing: review-first mode, tool allowlists, consent checks, a dark-pattern screen, stopping rules, and a full audit trail.
- We prove impact on a replayable batch with ground truth - gross recovered, incremental recovered (vs a holdout), net recovered, plus no-ops and escalations.

The Company: Razorpay

Razorpay Payments Private Limited is an RBI-authorised Payment Aggregator. Its platform lets merchants, schools, ecommerce businesses and other companies accept and disburse online payments, collect recurring payments, share invoices, and access business-finance products. Supported payment modes include credit cards, debit cards, netbanking, UPI and wallets. The critical point for us: a recovery system lives INSIDE a regulated payment flow, not beside an ordinary ecommerce CRM. That regulatory context shapes every design choice - what data we touch, what we are allowed to automate, and what needs merchant approval.

Razorpay reaches well beyond checkout. Businesses can manage marketplaces, automate bank transfers, collect recurring payments, share invoices, and access working-capital loans from one platform. RazorpayX adds business banking, corporate cards, payroll, vendor payments and API solutions. This breadth is exactly why a unified recovery view is possible: payment attempts, recurring obligations, receivables and operational cash flows all sit on the same platform.

Who owns the recovered money - the scope distinction that keeps us honest

In a merchant payment failure, the merchant carries the immediate commercial loss. We must NOT claim that every recovered rupee is Razorpay revenue. The strategic hypothesis is indirect: if Razorpay helps merchants convert more attempts into captures and collect more invoices, Razorpay's payment and finance platform becomes more valuable through stronger conversion, retention and differentiation. That platform-value hypothesis needs merchant cohort, pricing and retention data that public sources do not provide - so we measure merchant recovery and platform-adoption hypotheses separately, and never conflate them.

Razorpay surface	Revenue event	Our recovery value	Control boundary
Payment Gateway	Failed / incomplete payment attempt	Diagnose, retry where permitted, or reopen checkout	Never infer success from a failure notice - verify current payment and order state
Subscriptions	Failed auto-charge, pending subscription	Sequence retries, request customer action, or escalate	Use documented retry/webhook behaviour; do not invent terminal states
Payment Links / Invoices	Link sent but unpaid or near expiry	Controlled reminder, or a fresh approved link	Use amount, expiry, reference, customer and reminder controls
RazorpayX / business finance	Vendor, payroll, payout, cash ops	Context for merchant cash stress and escalation	Treat as context only unless explicit money-movement scopes exist

The design implication: make the merchant the policy owner and Razorpay the payment-state system of record. RevivalOS recommends and executes only approved actions, while the merchant always sees why a case was selected, what was attempted, and whether the money was actually captured.

The Buildathon: Razorpay AI Buildathon 2026

The buildathon's framing is 'Build. Show. Get hired.' - Razorpay reads the work, not the resume, judging how you think, build and solve problems. It is structured as an internship-hiring event (a monthly stipend reported at Rs 75,000, Bengaluru, in-person from September), open to current students without batch restriction. You pick one of five tracks, build a real working product (not a prototype), publish a public code repository with architecture documentation, and record a 5-minute pitch video explaining the problem, approach and results.

The five tracks

- 01 - AI Growth & Agentic Commerce: conversational in-app checkout, agent-readable catalog, upsell/cross-sell agent, campaign orchestrator.
- 02 - AI Risk Manager: fraud, risk and trust systems.
- 03 - AI Revenue Recovery: detect revenue at risk, choose the intervention, execute a bounded recovery workflow, prove money recovered. This is our track.
- 04 - AI Finance Controller: multi-source reconciliation, settlement Q&A agent, forward cash forecaster, tax-line matcher.
- 05 - Open Track: build anything that shows AI judgment.

Track 03 - the exact brief, in Razorpay's words

AI Revenue Recovery

- Find revenue that's slipping away and win it back.
- Build an agent that detects revenue at risk, determines the right intervention, and executes a bounded recovery workflow: from payment failures and checkout abandonment to overdue receivables.
- The bar: Don't just identify the problem. Show measured money recovered across a batch, with compliant escalation, stopping rules, and an audit trail.

Razorpay's own example directions for the track are: payment degradation to root cause to recovery action; checkout drop-off recovery; failed-subscription recovery; a B2B receivables chaser; a mandate retry sequencer; Hinglish voice recovery; and a promise-to-pay tracker. The bar is deliberately not 'a dashboard that says revenue is at risk.' It is measured money recovered, compliant escalation, stopping rules, and an audit trail.

How it is judged

Criterion	What Razorpay is really asking	How RevivalOS answers it
Problem taste	Did you pick something that actually matters?	Cross-surface leakage is a real, expensive, multi-step problem - not a toy.
Build quality	Does it run, is it structured, would you trust it?	Event-driven, reconciliation-first architecture with idempotency and a replay harness.
AI judgment	Right tool in the right place - and where you chose NOT to use one	AI drafts and classifies; deterministic code moves money. Abstention is a first-class output.
Failure recovery	Does it handle the messy path?	Duplicate/out-of-order events, stale state, refunds and disputes are demoed on purpose.

What we submit

- A public repository with clean structure and a one-command replay.
- An architecture diagram plus the event schema, the merchant policy file, the metric calculations, and a written list of what the agent refused to do.
- A 5-minute pitch video that walks the batch: ingest, classify, act, block a duplicate, escalate a high-value case, then correct a refund - ending on the recovery dashboard.

Why We Fit - and Why We Are Not a Clone

Razorpay's public Agent Studio already ships specialised agents: a Dispute Responder, a Subscription Recovery agent, and Abandoned Cart Conversion (partner-powered), alongside a Cashflow Forecaster and an RTO Shield. Agent Studio is built on Anthropic's Claude Agent SDK. That is a strategic signal, not a discouragement: rebuilding one cart reminder or one subscription retry would look like a narrow copy of an existing product. It raises the bar rather than removing it.

The differentiation in one sentence

- Where production Agent Studio has several separate specialist agents, RevivalOS is the ORCHESTRATOR above them: one merchant policy, one normalised case model, one decision engine that chooses among interventions, one ledger that attributes verified value, and one audit trail that explains why it stopped.

Six ways we think out of the box

- Recovery Control Plane, not a chatbot. The centrepiece is a case queue and a decision log, not a chat transcript. Judges see reasoning and reliability, not a clever conversation.
- The Intelligent No-Op. A confident decision to do nothing (a captured order, a conflicting webhook, a missing consent) is a scored, visible success - because the alternative is a double charge or a customer complaint. Almost no one demos restraint.
- Incremental, not gross. We run a treatment-vs-holdout experiment in the simulator so we can show money the agent actually caused, not payments that would have happened anyway. This is the single most credible number we can put on screen.
- Reconciliation-first. Every webhook is treated as a stale snapshot; we re-read live state before acting. We literally demo an old 'failed' event against a now-'paid' order and show the agent refuse to act.
- A visible Guardrail Console. Consent state, tool allowlist, dark-pattern screen and stop conditions are UI, not code comments - mirroring Razorpay's own Agent Studio principles.
- Explainability by construction. Every decision emits a reason code, a confidence, and a policy version - so the merchant can track, modify, and pinpoint exactly where a mistake happened and fix it.

Revenue Leakage: Four Core Lanes, One Adjacent Lane

We build a single normalised case model, but each recovery lane keeps its own trigger, action and accounting rules. The four core lanes are supported directly by Razorpay's documentation; the fifth is an adjacent reconciliation lane, not a retry lane.

Lane	Detection signal	First intervention	Recovery proof / stop
Payment failure	Failed status; order/payment state	Retry or reopen checkout only if eligible; else request a different permitted action	A later captured payment on the same order. Stop if captured, retried beyond policy, or decline is not recoverable
Abandoned checkout	Abandoned-cart webhook or a session with no successful payment	Personalised, consented return-to-checkout link	A later paid order attributable to the case. Verify payload fields in sandbox first
Failed subscription	Failed auto-charge, pending state, repeated-failure webhook	Retry sequence, payment-method update request, or human escalation	A successful charge within a defined attribution window. Never treat pending as unlimited-retry permission

Lane	Detection signal	First intervention	Recovery proof / stop
Overdue receivable	Unpaid invoice or link past a merchant due threshold	Reminder, link resend, promise-to-pay capture, or human queue	Payment applied to the original invoice/reference. Invoice event semantics need verification
Dispute / refund (adjacent)	Dispute or refund webhook	Evidence prep, merchant review, reconciliation	Dispute outcome or payment-not-reversed. Never count a refunded/disputed amount as recovery

The accounting rule that protects our credibility

We separate opportunity from outcome, always. 'At risk' is an estimate. 'Attempted' is an action. 'Recovered' is a verified capture or payment. 'Incremental recovered' is the amount above what would have happened without the agent. A refund, chargeback, duplicate capture or unrelated later payment must never inflate the result. This is the difference between a demo that survives scrutiny and one that does not.

The System: Six-Layer Architecture

The system is event-driven and reconciliation-first. AI is used AFTER the payment state is trustworthy, never before. Six layers, each preventing a specific class of failure.

Layer	What it does	Failure it prevents
1. Ingestion	Signed webhook receiver, raw-event store, replay queue	Forged events and untestable processing
2. State / Normaliser	Idempotent event key, ordering metadata, live-state read, reconciliation into one case model	Duplicate actions and stale-state recovery
3. Decision engine	Explainable score, reason code, policy version, confidence	Unreviewable or inconsistent choices
4. Bounded action layer	Tool allowlist, dry-run mode, retry budget, approval gate	Unbounded retries and unauthorised money movement
5. Outcome / ledger	Capture lookup, invoice linkage, refund/dispute exclusion	False claims of recovered revenue

Layer	What it does	Failure it prevents
6. Evidence / audit	Case timeline, message, tool arguments, human decisions, export	Failing the buildathon's audit requirement

The central reliability problem is that webhook payloads are snapshots. Razorpay explicitly notes a payment can move from authorized to captured quickly, and the authorized webhook reflects the earlier state; the captured event and order-paid event reflect the later truth. So the agent must be able to say: 'the event was failed, but the current order is paid, so no recovery action is allowed.' That single sentence, demoed live, is worth more than any chat feature.

Where AI is used - and where it is forbidden

A deterministic state machine owns payment truth. The LLM is used only for bounded tasks: summarising a failure, selecting among approved playbooks, drafting a customer message, extracting a promise-to-pay date, or explaining a human-queue case. If the model cannot identify the entity, current state, allowed action or consent status, it abstains and escalates.

Decision	AI may do	AI may NOT do without explicit policy + approval
Eligibility	Summarise evidence, assign a reason code	Declare a payment recovered from a webhook alone
Prioritisation	Rank cases by expected value and urgency	Change merchant thresholds silently
Retry	Recommend timing within a fixed budget	Retry hard or ambiguous cases indefinitely
Communication	Draft a clear message in an approved language and channel	Use false urgency, shame, threats, or an unapproved offer
Receivables	Extract a reply, promise date, or dispute reason	Admit liability, waive debt, or alter invoice terms
Escalation	Build a concise evidence packet	Bypass a merchant approval gate

The AI Agent Layer - How Many, and Why Each Exists

RevivalOS runs a small crew of bounded agents behind one orchestrator. Each has a single job, a fixed tool allowlist, and an abstention path. This mirrors Agent Studio's 'each agent is a specialist you hire for one job' model while keeping our orchestrator as the differentiator. Seven agents total.

Agent	Its one job	Why it exists / where it acts
Orchestrator	Route each verified case to the right lane and playbook	The differentiator: one policy across all surfaces
Triage & Classifier	Read status + error code, assign failure class + reason code + confidence	Turns raw events into explainable, prioritised cases
Prioritiser	Rank by expected net recovery (value x probability x urgency)	Focuses human and automated attention where money is
Retry Strategist	Recommend retry timing within the merchant's fixed budget	Payment + subscription lanes; never exceeds the budget
Communicator	Draft a consented, on-brand, non-manipulative message	Abandoned-cart + receivables lanes; passes the dark-pattern screen
Receivables Reader	Extract reply intent, promise-to-pay date, dispute reason	B2B receivables chaser and promise-to-pay tracker
Escalation Builder	Assemble a concise evidence packet for a human	High-value, low-confidence, or conflicting cases

Expected-value scoring

- Each eligible case is scored as expected net recovery = recoverable amount x estimated recovery probability x urgency weight, minus expected cost and harm risk.
- The probability model starts as rules and can later learn from outcomes. Stripe's public Smart Retries (AI-chosen retry timing within a bounded attempt policy) is a design REFERENCE only - not evidence Razorpay exposes the same signals.
- HighRadius' behaviour-based collections (a reported Ferrero deployment cut DSO 28% and days-delinquent 67%, saving 1,000+ hours/yr) is vendor-reported, not a Razorpay benchmark - but it proves the mechanism: prioritise attention and personalise the next action instead of increasing message volume.

The App, Screen by Screen

This is how RevivalOS actually looks and works to a merchant operator - opening the app, moving through every screen, and understanding what each feature does and why it earns its place. Read this as the product tour.

Screen 0 - Login & workspace lock

The operator signs in with their Razorpay-scoped credentials. The workspace is locked to a single merchant account and a single environment (Test or Live), shown as a hard banner. In the buildathon build this is Test mode only, by design - we never touch production money. A visible environment lock is a trust feature: an operator can never accidentally act on live customers from a sandbox session.

Screen 1 - Home / Recovery Cockpit

The landing screen is the cockpit, not a chat box. Top strip: four live counters - At-Risk Value, Recovered (verified), Incremental Recovered, and Net Recovered. Below them, four lane tiles (Payments, Checkout, Subscriptions, Receivables) each showing open cases and money in play. A right rail shows what the agents did in the last hour and what is waiting for human approval. The point of Home is that in five seconds a merchant knows how much money is at stake, how much came back, and whether anything needs them.

- Why it exists: the brief rewards measured money recovered - so money is the first thing on screen, split into gross, incremental and net.
- Why the split: it stops us from over-claiming. Gross is the headline; incremental is the honest number; net subtracts refunds, discounts, fees and comms cost.

Screen 2 - Case Queue

The working surface. A prioritised, filterable list of every open case across all four lanes, ranked by expected net recovery. Each row shows: case ID, lane, amount, age, failure class, current verified state, the proposed action, its confidence, and approval status. Filters by lane, value band, age, consent state and 'needs approval.' This is where an operator spends their day.

- Why ranked, not chronological: attention is the scarce resource; the highest-value recoverable cases surface first.
- Why 'current verified state' is a column: it is the reconciliation guarantee made visible - a case whose order is now paid shows as a no-op, right in the list.

Screen 3 - Case Detail & Decision Log

Open any case to see its full story: the original event, every later state change, the reconciled current state, the classifier's reason code and confidence, the chosen playbook, every tool call with its arguments, any drafted message, the customer's response, the eventual outcome, and the stop reason. This is the audit trail as a screen - one case, fully explainable, top to bottom.

- Why it matters for judging: audit completeness is an explicit metric. A case with source event, decision reason, tool result, outcome and stop reason is a provably traceable case.
- Why the operator loves it: this is exactly where they track what the AI did, modify a decision, and pinpoint where a mistake happened - then fix it.

Screen 4 - Review-First Approvals

A dedicated inbox for actions the agent has PREPARED but not sent - drafted messages, proposed retries, link regenerations, escalations. The operator approves, edits, or rejects. Irreversible actions always land here. Review-first mode means the agent can do all the thinking and preparation without ever acting until a human says go - and the merchant chooses which action classes are auto-approved versus held.

Screen 5 - Guardrail Console

The safety cockpit, made visible. It shows, per action: the tool allowlist (what this agent is even allowed to call), the consent state of the customer, the dark-pattern screen result on any drafted message, the per-case retry budget and how much is spent, and the active stop conditions. A blocked tool call and a refused manipulative message both appear here as events - safety you can watch working.

- Why it is a screen and not code: Razorpay's Agent Studio principles - review-first, merchant-defined permissions, scope checks, approval for irreversible actions, consent before outbound contact, dark-pattern screening, full logging - are product features we surface, not implementation details we hide.
- The dark-pattern screen actively refuses false urgency, confirm-shaming, manufactured scarcity and subscription traps.

Screen 6 - Recovery Ledger

The immutable money record. Every observation, decision, approval, tool call, result, recovery amount, reversal and stop reason, in order, append-only. When a refund or dispute lands, the ledger corrects the recovery total automatically - and you can watch it happen. This is the single source of truth behind every number on Home.

Screen 7 - Merchant Policy Editor

Where the merchant owns the rules. Retry budget, amount thresholds, due-age thresholds, allowed channels, consent rules, approval rules, and the maximum discount the Communicator may ever offer (the agent picks from merchant-authorised offers only - it never invents a discount). Every change is versioned, and every decision on a case records which policy version produced it. This makes the agent configurable rather than hard-coded, and makes any past decision reproducible.

Screen 8 - Replay & Experiment Studio

The demo weapon. Load a batch of synthetic or anonymised cases with known ground-truth outcomes and press Replay. Watch the whole system run: ingest a failed payment, classify it, take a bounded action, inject a duplicate/out-of-order event and see reconciliation block a second charge, replay an abandoned cart with a consented message, process a pending subscription, route a high-value overdue receivable to review, then apply a refund and watch the ledger correct itself. A treatment-vs-holdout toggle shows incremental recovery - money the agent actually caused.

- Why it exists: it makes the entire pitch repeatable without live financial access, and it is where the incremental number - our most credible claim - is generated.
- Why holdout matters: the holdout sends nothing; it establishes how many cases recover naturally, so we never credit the agent for payments that would have happened anyway.

Screen 9 - Metrics & Audit Export

The scoreboard and the receipts. Every metric below, plus a one-click audit export of the full case set for review. If numbers are simulated, they are labelled simulated, with the exact fixtures and calculation shown - never a model's success probability presented as recovered revenue.

Metric	Definition	Why it matters
At-risk value	Sum of eligible unpaid amounts at decision time	Measures opportunity, not success
Eligible rate	Eligible cases / all detected cases	Shows policy discipline
Recovery rate	Verified recovered / eligible cases	Shows operational effectiveness
Gross recovered	Verified captured/paid amount linked to an action	The direct outcome
Incremental recovered	Treatment outcome minus matched holdout	Avoids credit for payments that would happen anyway

Metric	Definition	Why it matters
Net recovered	Incremental minus refunds, discounts, fees, comms cost	Aligns optimisation with economics
False-action rate	Actions later found unnecessary/duplicated/inconsistent	Measures financial risk
Customer-harm rate	Complaints, opt-outs, failed consent per contact	Measures trust cost
Automation vs escalation	Cases completed safely vs sent to human review	Shows where AI judgment is useful
Audit completeness	Cases with event + reason + result + outcome + stop	Proves traceability

Track, Modify, and Fix - What the Company Gets

A recurring ask in your brief: the portal must let the company easily track what the agents did, easily modify it, and easily identify and resolve mistakes. RevivalOS answers this by construction.

- Track: the Decision Log and Recovery Ledger record every step of every case, append-only, with reason codes, confidence, tool arguments and outcomes - so 'what did the AI do here' is always one click away.
- Modify: the Merchant Policy Editor and Review-First Approvals let the operator change thresholds, edit a drafted action, or reject a decision before it acts - and re-run the case under the new policy.
- Identify mistakes: the False-Action Rate and Customer-Harm Rate surface where the agent went wrong; every decision carries the policy version that produced it, so a bad pattern is traceable to a specific rule.
- Resolve: fix the policy, replay the affected batch, and confirm the metric moved - a closed loop from mistake to fix to proof, all inside the product.

Safety, Compliance, and Merchant Control

In financial workflows, safety IS product quality - and it belongs in the demo. Razorpay is PCI-DSS certified and its Agent Studio infrastructure adheres to India's DPDPA. So RevivalOS avoids raw card numbers, CVV, authentication secrets and unnecessary personal data entirely: we use tokenised references, masked identifiers, synthetic fixtures and scoped test credentials. Any external AI platform is treated as a separate data processor, consistent with Razorpay's privacy stance that AI integrations are optional and process only what the action needs.

Risk	Default control	What the pitch shows
Duplicate / stale event	Idempotency key + live-state reconciliation	A captured order causes a logged no-op
Excessive retry	Per-case attempt count, time window, merchant budget	The agent stops and explains why
Unauthorised action	Tool allowlist, scope check, review-first, explicit approval	A blocked tool call visible in the audit trail
Harmful communication	Consent check, approved templates, dark-pattern classifier, opt-out	The system refuses manipulative wording
Privacy / credential leak	Synthetic data, token references, secret manager, minimal context	The model prompt contains no card data
Refund / dispute confusion	Separate case types + accounting exclusions	A refund is removed from recovered value

On the Razorpay MCP Server: it is a real Model Context Protocol integration between Razorpay's payment APIs and AI tools (Payments, Orders, Payment Links, Refunds, QR codes, Settlements), usable from tools like Claude Desktop, Cursor and VS Code. We treat MCP as an OPTIONAL adapter and request Razorpay's approved tool schema rather than assuming every API operation should be exposed to an agent.

What We Need From Razorpay Before Building

The biggest project risk is not model quality - it is the lack of a trustworthy action-and-outcome contract. Without eventual outcomes we can rank risk but cannot honestly claim recovery. Ask for these in priority order.

Priority	What to request	Why we need it
P0	Test credentials, webhook secret, event fixtures, API docs, sandbox replay	Build and verify the event loop without touching production money
P0	Live-state reads + approved action endpoints	Reconcile stale events and execute only real, permitted actions
P0	Ground-truth outcomes or a labelled simulator	Calculate recovered and incremental value honestly
P1	Merchant policy schema (budgets, thresholds, channels, consent, approvals)	Make the agent configurable, not hard-coded

Priority	What to request	Why we need it
P1	Synthetic customer/merchant profiles + failure taxonomy	Test personalisation without exposing PII
P1	Communication sandbox (email / WhatsApp / SMS)	Demonstrate consent, templates, opt-out and delivery
P2	Mentor feedback on target segment + constraints	Focus scope, avoid duplicating an Agent Studio feature
P2	Approved MCP tool schema, if MCP is used	Prevent accidental exposure of broad payment operations

If Razorpay can't provide live APIs or outcomes

- Build a high-fidelity adapter interface with recorded fixtures and label the result clearly as simulation. That is far better than fabricating production impact - and it still demonstrates every capability the track rewards.

Build Plan and Scope Discipline

Build the common case model and the replay simulator FIRST, then activate each lane's adapter only once its event and action contract is verified. The public documentation is richer for payment and subscription behaviour than for exact abandoned-cart and invoice payloads, so we sequence by evidence, not ambition.

Phase	What we ship	Proof it works
1. Foundation	Case model, ingestion, idempotency, live-state reconciliation, ledger	A duplicate event produces a logged no-op
2. Payment-failure core	Classifier, eligibility engine, bounded retry, decision log	A stale 'failed' event on a now-paid order is refused
3. Abandoned checkout	Consented Communicator + return link, attribution	A later paid order is credited to the case
4. Subscriptions + receivables	Retry sequencer, promise-to-pay tracker, human queue	A high-value overdue case routes to review
5. Proof + polish	Replay studio, holdout experiment, metrics, audit export, pitch	Incremental + net recovery shown on a batch

Final deliverables checklist

- Public repository with clean structure and a one-command replay.
- Architecture diagram, event schema, merchant policy file, metric calculations.
- A replayable batch with ground truth showing gross, incremental and net recovery, plus no-ops and escalations.
- A written 'what the agent refused to do' section.
- A 5-minute pitch that ends on the recovery dashboard, not a chat transcript.

Synthesis

The winning strategy combines four sources without importing any one's assumptions uncritically: Razorpay's stateful payment infrastructure as the system of record; Stripe-style decision optimisation as a mechanism reference; enterprise AR prioritisation (HighRadius) as evidence that personalising attention beats increasing volume; and Razorpay's own merchant-control philosophy as the guardrail model. The resolution to every tension - automation vs control, breadth vs evidence, gross vs incremental - is the same: automation inside a visible policy envelope, with attempt budgets, abstention, and an honest ledger.

So we build a Recovery Control Plane: payment-failure recovery as the reliable core, abandoned-checkout recovery as the visible growth case, subscriptions and payment links as extensions, and disputes/refunds as a reconciliation lane. Deterministic code owns payment truth. AI owns bounded reasoning and drafts. Merchants own permissions and irreversible decisions. That design is useful to Razorpay even if the production product eventually splits the work into several specialist agents - which is exactly why it stands out.